Java 8 Features

Complete Interview Preparation Guide

This document covers all major Java 8 features with:
✓ Simple Explanations
✓ Easy Examples
✓ Interview Questions & Answers

Quick Introduction

Java 8 launched on 18th March 2014 and it was a next major release after jdk5. It came up with large set of scintillating features that has won the attention of most of the java programmers. It has enhanced various java components like runtime environment, compiler, lexical parser, memory management, command line tools and many more. Java 8 will improve programmer's coding experience with its enticed features of declarative programming, passing code as an argument, method reference, optional for handling null etc. It will assure you to write codes that will be more precise, objective driven and highly readable.

Passing methods as parameter removes verbosity from the code and in fact increases reusability, Streams helps in writing SQL like syntaxes, parallelization that is almost

free:- speed of the execution with efficient use of modern computers having multicore processors, handling nullable values using Optional and many more. Initially stuffs will be little confusing but once you used to it, you will be reluctant to write code with out using it. Let’s look into the below usecase and understand why java 8 is different then all the releases.

Table of Contents

1. Lambda Expressions
2. Functional Interfaces
3. Stream API
4. Optional Class
5. Method References
6. Default and Static Methods in Interfaces
7. Date and Time API
8. forEach Method

1. Lambda Expressions

Brief Explanation

Lambda expressions are a way to write short, anonymous functions (functions without a name) in Java. They make your code cleaner and easier to read by reducing the amount of code you need to write.

Summary

- Lambda expressions are anonymous functions
- They help in writing less code
- They enable functional programming in Java
- Syntax: (parameters) -> expression or (parameters) -> { statements; }
- Used primarily with Functional Interfaces

Detailed Description

Before Java 8, if you wanted to pass behavior (like sorting logic) to a method, you had to create an anonymous class, which required a lot of code. Lambda expressions solve this problem by allowing you to write the same functionality in just one line.

Think of lambda expressions as a shortcut for writing methods. Instead of creating a whole class with a method, you can write a small function inline.

Lambda Syntax

```
(parameters) -> expressionor(parameters) -> { statements; }
```

Simple Examples

Example 1: Without Parameters

Old Way (before Java 8):

```
Runnable r = new Runnable() {    @Override    public void run() {
System.out.println("Hello World");    }};
```

New Way (with Lambda):

```
Runnable r = () -> System.out.println("Hello World");
```

Example 2: With One Parameter

```
List<String> names = Arrays.asList("Tom", "Jerry",
"Spike");names.forEach(name -> System.out.println(name));
```

This prints each name in the list. The 'name' is the parameter that represents each item in the list.

Example 3: With Two Parameters

```
// Adding two numbers(int a, int b) -> a + b// Or you can skip the
type(a, b) -> a + b
```

Java can figure out the types automatically, so you don't always need to write them.

Interview Questions and Answers

Q1: What is a Lambda Expression in Java 8?

Answer: A Lambda Expression is an anonymous function (a function without a name) that you can use to write cleaner and more concise code. It allows you to pass behavior as a parameter to methods.

Q2: What is the syntax of Lambda Expression?

Answer: The syntax is (parameters) -> expression or (parameters) -> { statements; }. For example: (x, y) -> x + y

Q3: Can you use Lambda expressions with any interface?

Answer: No. Lambda expressions can only be used with Functional Interfaces (interfaces that have only one abstract method).

Q4: What are the advantages of Lambda Expressions?

- Less code - You can write functions in one line
- More readable - Code is cleaner and easier to understand
- Enables functional programming in Java
- Can be passed as arguments to methods

Q5: Can a Lambda expression throw an exception?

Answer: Yes, but if the functional interface's method declares a checked exception, the lambda must either handle it or declare it in the method signature.

2. Functional Interfaces

Brief Explanation

A Functional Interface is an interface that has exactly one abstract method. Java 8 introduced these interfaces to work with Lambda expressions. They can have multiple default or static methods, but only one abstract method.

Summary

- Contains exactly ONE abstract method
- Can have multiple default and static methods
- Used with Lambda expressions
- Marked with @FunctionalInterface annotation (optional but recommended)
- Examples: Runnable, Callable, Comparator, Predicate, Function, Consumer, Supplier

Detailed Description

Functional interfaces are the foundation of functional programming in Java 8. They allow you to use lambda expressions and method references. Java provides many built-in functional interfaces in the java.util.function package.

The @FunctionalInterface annotation is optional, but it's good practice to use it. It tells the compiler that the interface should have only one abstract method, and it will give an error if you accidentally add more.

Simple Examples

Example 1: Creating Your Own Functional Interface

```
@FunctionalInterfaceinterface Calculator {    int calculate(int a, int
b);}// Using it with LambdaCalculator add = (a, b) -> a +
b;System.out.println(add.calculate(5, 3)); // Output: 8
```

Example 2: Predicate (Built-in Functional Interface)

Predicate is used to test a condition and return true or false.

```
Predicate<Integer> isEven = num -> num % 2 ==
0;System.out.println(isEven.test(4)); // Output:
trueSystem.out.println(isEven.test(5)); // Output: false
```

Example 3: Function (Built-in Functional Interface)

Function takes an input and returns an output.

```
Function<String, Integer> getLength = str ->
str.length();System.out.println(getLength.apply("Hello")); // Output:
5
```

Example 4: Consumer (Built-in Functional Interface)

Consumer takes an input but doesn't return anything.

```
Consumer<String> print = str ->
System.out.println(str);print.accept("Hello World"); // Output: Hello
World
```

Example 5: Supplier (Built-in Functional Interface)

Supplier doesn't take any input but returns a value.

```
Supplier<String> getMessage = () -> "Welcome to Java
8";System.out.println(getMessage.get()); // Output: Welcome to Java 8
```

Common Built-in Functional Interfaces

Interface	Method	Use
Predicate<T>	boolean test(T t)	Test a condition
Function<T, R>	R apply(T t)	Transform input to output
Consumer<T>	void accept(T t)	Perform action on input
Supplier<T>	T get()	Supply a value

Interview Questions and Answers

Q1: What is a Functional Interface?

Answer: A Functional Interface is an interface with exactly one abstract method. It can have multiple default and static methods. Examples include Runnable, Callable, Comparator, and interfaces in the java.util.function package.

Q2: What is the purpose of @FunctionalInterface annotation?

Answer: The @FunctionalInterface annotation is optional but recommended. It tells the compiler to check that the interface has only one abstract method. If you accidentally add more than one abstract method, the compiler will give an error.

Q3: Can a Functional Interface have more than one method?

Answer: It can have only ONE abstract method, but it can have multiple default methods and static methods. It can also have methods from Object class like equals(), hashCode(), toString().

Q4: Name some commonly used built-in Functional Interfaces.

- Predicate<T> - Tests a condition
- Function<T, R> - Transforms input to output
- Consumer<T> - Performs action on input
- Supplier<T> - Supplies a value
- BiFunction<T, U, R> - Takes two inputs and returns output

Q5: What is the difference between Predicate and Function?

Answer: Predicate always returns a boolean value (true/false) using the test() method. Function returns any type of value using the apply() method. Predicate is used for testing conditions, while Function is used for transforming data.

3. Stream API

Brief Explanation

Stream API is a powerful feature in Java 8 that helps you process collections of data (like lists, sets, arrays) in a functional way. Instead of writing loops, you can use streams to filter, map, sort, and perform other operations on data in a clean and readable manner.

Summary

- Process collections in a functional style
- No need to write traditional for loops
- Makes code more readable and concise
- Supports parallel processing for better performance
- Common operations: filter, map, reduce, collect, forEach, sorted

Detailed Description

A Stream is not a data structure. It's a pipeline through which data flows and gets processed. You create a stream from a collection, apply operations to it, and then collect the results.

Stream operations are divided into two types:

- **Intermediate Operations:** These return a new stream and are lazy (they don't execute until a terminal operation is called). Examples: filter(), map(), sorted()
- **Terminal Operations:** These produce a result or side-effect and end the stream. Examples: collect(), forEach(), count(), reduce()

Simple Examples

Example 1: Filter - Get Even Numbers

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);List<Integer>
evenNumbers = numbers.stream()    .filter(n -> n % 2 == 0)
.collect(Collectors.toList());System.out.println(evenNumbers); //
Output: [2, 4, 6]
```

Example 2: Map - Convert to Uppercase

```
List<String> names = Arrays.asList("alice", "bob",
"charlie");List<String> upperNames = names.stream()
.map(String::toUpperCase)
.collect(Collectors.toList());System.out.println(upperNames); //
Output: [ALICE, BOB, CHARLIE]
```

Example 3: Sorted - Sort Names

```
List<String> names = Arrays.asList("Charlie", "Alice",
"Bob");List<String> sortedNames = names.stream()    .sorted()
.collect(Collectors.toList());System.out.println(sortedNames); //
Output: [Alice, Bob, Charlie]
```

Example 4: Count - Count Elements

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");long
count = names.stream().count();System.out.println(count); // Output: 3
```

Example 5: Reduce - Sum of Numbers

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);int sum =
numbers.stream()    .reduce(0, (a, b) -> a +
b);System.out.println(sum); // Output: 15
```

Example 6: Chaining Multiple Operations

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7,
8);List<Integer> result = numbers.stream()    .filter(n -> n % 2 == 0)
// Get even numbers    .map(n -> n * 2)           // Double them
.collect(Collectors.toList());System.out.println(result); // Output:
[4, 8, 12, 16]
```

Common Stream Operations

Operation	Description
filter()	Selects elements that match a condition
map()	Transforms each element to something else
sorted()	Sorts elements in natural or custom order
distinct()	Removes duplicate elements
limit()	Limits the number of elements
collect()	Collects stream elements into a collection (List, Set, etc.)
forEach()	Performs an action for each element
count()	Returns the number of elements
reduce()	Combines all elements into a single result

Interview Questions and Answers

Q1: What is Stream API in Java 8?

Answer: Stream API is a feature that allows you to process collections of data in a functional way. It provides methods like filter, map, reduce to perform operations on data without writing traditional loops.

Q2: What is the difference between Collection and Stream?

Answer: Collection is a data structure that stores elements in memory. Stream is not a data structure; it's a pipeline for processing data. Collections can be iterated multiple

times, but streams can only be used once. Streams support lazy evaluation and can work with infinite data.

Q3: What are Intermediate and Terminal operations in Streams?

Answer:

- **Intermediate Operations:** Return a new stream and are lazy (don't execute until terminal operation is called). Examples: filter(), map(), sorted(), distinct()
- **Terminal Operations:** Produce a result and end the stream. Examples: collect(), forEach(), count(), reduce()

Q4: Can you reuse a Stream?

Answer: No. Once a terminal operation is performed on a stream, it cannot be reused. You need to create a new stream if you want to perform more operations.

Q5: What is the difference between map() and flatMap()?

Answer: map() transforms each element to another element (one-to-one mapping). flatMap() transforms each element to a stream and then flattens all streams into one (one-to-many mapping). Example: If you have a list of lists, flatMap() can flatten it into a single list.

Q6: What is parallel stream?

Answer: A parallel stream divides the data into multiple chunks and processes them in parallel using multiple threads, which can improve performance for large datasets. You can create a parallel stream using parallelStream() method.

4. Optional Class

Brief Explanation

Optional is a container class that may or may not contain a non-null value. It helps you avoid NullPointerException by forcing you to handle the absence of a value explicitly. Instead of returning null, methods can return an Optional object.

Summary

- Helps avoid NullPointerException
- Container that may or may not hold a value
- Makes null-handling explicit and cleaner
- Common methods: of(), ofNullable(), empty(), isPresent(), orElse(), orElseGet(), ifPresent()
- Best used as return type of methods, not as class fields

Detailed Description

Before Java 8, when a method could return null, you had to check for null every time. If you forgot to check, you would get a NullPointerException at runtime. Optional forces you to handle the case when a value might be absent.

Optional is like a box that either contains a value or is empty. You can check if the box has a value and handle both cases appropriately.

Simple Examples

Example 1: Creating Optional Objects

```
// Optional with a valueOptional<String> optional1 =
Optional.of("Hello");// Optional with null (will throw exception)//
Optional<String> optional2 = Optional.of(null); // Don't do this!//
Optional that can handle nullOptional<String> optional3 =
Optional.ofNullable(null);// Empty OptionalOptional<String> optional4
= Optional.empty();
```

Example 2: Checking if Value is Present

```
Optional<String> optional = Optional.of("Java 8");if
(optional.isPresent()) {    System.out.println(optional.get());} else
{    System.out.println("No value present");}// Output: Java 8
```

Example 3: Using orElse()

```
Optional<String> optional = Optional.empty();String result =
optional.orElse("Default Value");System.out.println(result); //
Output: Default Value
```

Example 4: Using ifPresent()

```
Optional<String> optional =
Optional.of("Hello");optional.ifPresent(value ->
System.out.println(value));// Output: Hello
```

Example 5: Using map() with Optional

```
Optional<String> optional = Optional.of("hello");Optional<String>
upperCase =
optional.map(String::toUpperCase);System.out.println(upperCase.get());
// Output: HELLO
```

Example 6: Practical Use Case

```
// Old way (before Java 8)public String getCustomerName(Long id) {
Customer customer = findCustomer(id);    if (customer != null) {
return customer.getName();    }    return "Unknown";}// New way (with
Optional)public String getCustomerName(Long id) {    return
findCustomer(id)        .map(Customer::getName)
.orElse("Unknown");}private Optional<Customer> findCustomer(Long id) {
// returns Optional<Customer>}
```

Common Optional Methods

Method	Description
of(value)	Creates Optional with non-null value (throws exception if null)
ofNullable(value)	Creates Optional that can handle null values
empty()	Creates an empty Optional
isPresent()	Returns true if value is present, false otherwise
get()	Returns the value (throws exception if empty)
orElse(default)	Returns value if present, otherwise returns default
orElseGet(supplier)	Returns value if present, otherwise calls supplier function
ifPresent(consumer)	Executes consumer if value is present

Interview Questions and Answers

Q1: What is Optional class in Java 8?

Answer: Optional is a container class that may or may not contain a non-null value. It helps avoid NullPointerException by making null-handling explicit. Instead of returning null, methods can return Optional to indicate that a value might be absent.

Q2: How do you create an Optional object?

Answer: You can create Optional using: Optional.of(value) for non-null values, Optional.ofNullable(value) for values that might be null, and Optional.empty() for an empty Optional.

Q3: What is the difference between orElse() and orElseGet()?

Answer: orElse() always executes and returns the default value, even if the Optional contains a value. orElseGet() takes a Supplier and only executes it if the Optional is empty. orElseGet() is more efficient when creating the default value is expensive.

Q4: Should we use Optional as a field in a class?

Answer: No. Optional is designed to be used as a return type of methods, not as a field type. Using Optional as a field adds unnecessary overhead and complexity. It's better to use null for fields and handle it appropriately.

Q5: How does Optional help avoid NullPointerException?

Answer: Optional forces you to explicitly handle the case when a value is absent. Instead of directly accessing a potentially null value, you must check if it's present using isPresent() or use safe methods like orElse(), orElseGet(), or ifPresent(). This makes your code more robust and prevents unexpected null reference errors.

5. Method References

Brief Explanation

Method References are a shorthand way to write lambda expressions that simply call an existing method. Instead of writing a lambda like (x) -> someMethod(x), you can write it as ClassName::someMethod. They make code more readable and concise.

Summary

- Shorthand for lambda expressions
- Use :: (double colon) operator
- Makes code more readable
- Four types: Static method, Instance method, Constructor, Array constructor

Detailed Description

When a lambda expression just calls a single method, you can replace it with a method reference. This makes the code cleaner and easier to read. Method references don't do anything new; they're just a more compact way of writing lambdas.

Types of Method References

1. Reference to Static Method

Syntax: ClassName::staticMethodName

```
// Lambda expressionFunction<String, Integer> parseFunc = s ->
Integer.parseInt(s);// Method referenceFunction<String, Integer>
parseFunc =
Integer::parseInt;System.out.println(parseFunc.apply("123")); //
Output: 123
```

2. Reference to Instance Method of a Particular Object

Syntax: object::instanceMethodName

```
String str = "Hello World";// Lambda expressionSupplier<String>
supplier = () -> str.toUpperCase();// Method referenceSupplier<String>
supplier = str::toUpperCase;System.out.println(supplier.get()); //
Output: HELLO WORLD
```

3. Reference to Instance Method of an Arbitrary Object

Syntax: ClassName::instanceMethodName

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");//
Lambda expressionnames.forEach(name -> System.out.println(name));//
Method referencenames.forEach(System.out::println);
```

4. Reference to Constructor

Syntax: ClassName::new

```
// Lambda expressionSupplier<List<String>> listSupplier = () -> new
ArrayList<>();// Method referenceSupplier<List<String>> listSupplier =
ArrayList::new;List<String> list = listSupplier.get();
```

More Examples

Example 1: Sorting with Method Reference

```
List<String> names = Arrays.asList("Charlie", "Alice", "Bob");//
Lambdanames.sort((a, b) -> a.compareTo(b));// Method
referencenames.sort(String::compareTo);System.out.println(names); //
Output: [Alice, Bob, Charlie]
```

Example 2: Using with Stream

```
List<String> words = Arrays.asList("apple", "banana", "cherry");//
Convert to uppercase using method referenceList<String> upperWords =
words.stream()    .map(String::toUpperCase)
.collect(Collectors.toList());System.out.println(upperWords); //
[APPLE, BANANA, CHERRY]
```

Interview Questions and Answers

Q1: What is a Method Reference in Java 8?

Answer: A Method Reference is a shorthand notation for calling an existing method. It uses the :: (double colon) operator and makes lambda expressions more concise. For example, instead of x -> System.out.println(x), you can write System.out::println.

Q2: What are the types of Method References?

- Static method reference: ClassName::staticMethod
- Instance method reference of a particular object: object::instanceMethod
- Instance method reference of an arbitrary object: ClassName::instanceMethod
- Constructor reference: ClassName::new

Q3: Can you give an example of constructor reference?

Answer: Constructor references are used to create new objects. For example: Supplier<List<String>> listSupplier = ArrayList::new; creates a supplier that returns a new ArrayList when called.

Q4: When should you use Method References instead of Lambda?

Answer: Use method references when your lambda expression simply calls an existing method without any additional logic. This makes code more readable. However, if you need to add extra logic or manipulate parameters, stick with lambda expressions.

6. Default and Static Methods in Interfaces

Brief Explanation

Java 8 allows interfaces to have default methods (with implementation) and static methods. Before Java 8, interfaces could only have abstract methods. This feature helps in adding new methods to interfaces without breaking existing implementations.

Summary

Default Methods:

- Have implementation in interface
- Use 'default' keyword
- Can be overridden by implementing classes
- Help maintain backward compatibility

Static Methods:

- Have implementation in interface
- Use 'static' keyword
- Called using interface name
- Cannot be overridden

Detailed Description

Before Java 8, if you wanted to add a new method to an interface, all classes implementing that interface would have to implement the new method, which could break existing code. Default methods solve this problem by providing a default implementation that all implementing classes can use or override.

Static methods in interfaces are utility methods that belong to the interface itself, not to the implementing classes.

Simple Examples

Example 1: Default Method

```
interface Vehicle {    void start();        // Default method
default void honk() {        System.out.println("Honk! Honk!");
}}class Car implements Vehicle {    @Override    public void start() {
System.out.println("Car started");    }    // No need to implement
honk() - uses default}Car car = new Car();car.start();  // Output: Car
startedcar.honk();   // Output: Honk! Honk!
```

Example 2: Overriding Default Method

```
class Bike implements Vehicle {    @Override    public void start() {
System.out.println("Bike started");    }        @Override    public
void honk() {        System.out.println("Beep! Beep!");    }}Bike bike
= new Bike();bike.start();  // Output: Bike startedbike.honk();   //
Output: Beep! Beep!
```

Example 3: Static Method in Interface

```
interface MathOperations {    // Static method    static int add(int
a, int b) {        return a + b;    }        static int multiply(int
a, int b) {        return a * b;    }}// Calling static methodsint sum
= MathOperations.add(5, 3);int product = MathOperations.multiply(4,
2);System.out.println("Sum: " + sum);         // Output: Sum:
8System.out.println("Product: " + product); // Output: Product: 8
```

Example 4: Combining Default and Static Methods

```
interface Logger {    // Default method    default void log(String
message) {        System.out.println("Log: " + message);    }
// Static method    static void info(String message) {
System.out.println("INFO: " + message);    }}class MyLogger implements
Logger {    // Can use default log() or override it}MyLogger logger =
new MyLogger();logger.log("Application started");  // Uses default
methodLogger.info("System initialized");  // Calls static method
```

Interview Questions and Answers

Q1: What are default methods in interfaces?

Answer: Default methods are methods in interfaces that have a default implementation. They are declared using the 'default' keyword. Implementing classes can use the default implementation or override it. This feature helps add new methods to interfaces without breaking existing implementations.

Q2: Why were default methods introduced in Java 8?

Answer: Default methods were introduced to maintain backward compatibility. Before Java 8, if you added a new method to an interface, all implementing classes had to implement it, which would break existing code. With default methods, you can add new methods with default implementations without affecting existing code.

Q3: Can we have static methods in interfaces?

Answer: Yes. Java 8 allows static methods in interfaces. These methods belong to the interface itself and must be called using the interface name (InterfaceName.methodName()). They cannot be overridden by implementing classes.

Q4: What happens if a class implements two interfaces with the same default method?

Answer: This creates a conflict called the 'Diamond Problem'. The class must override the method and provide its own implementation. You can also explicitly call one of the default methods using InterfaceName.super.methodName().

Q5: Can default methods be abstract?

Answer: No. Default methods must have an implementation. The whole point of default methods is to provide a default implementation that can be used or overridden. Abstract methods are methods without implementation.

7. Date and Time API

Brief Explanation

Java 8 introduced a new Date and Time API in the java.time package to replace the old java.util.Date and java.util.Calendar classes. The new API is easier to use, thread-safe, and provides better methods for handling dates, times, and time zones.

Summary

- Package: java.time
- Immutable and thread-safe
- Main classes: LocalDate, LocalTime, LocalDateTime, ZonedDateTime
- Period and Duration for date/time differences
- Better than old Date and Calendar classes

Detailed Description

The old Date and Calendar classes had many problems: they were not thread-safe, the month numbering started from 0 (January = 0), and they were mutable (you could change their values). The new Date and Time API solves all these problems.

The API is divided into two main parts: Local (without time zone) and Zoned (with time zone).

Main Classes

Class	Description
LocalDate	Represents date only (year, month, day)
LocalTime	Represents time only (hour, minute, second)
LocalDateTime	Represents both date and time (no time zone)
ZonedDateTime	Represents date and time with time zone
Period	Represents date-based difference (years, months, days)
Duration	Represents time-based difference (hours, minutes, seconds)

Simple Examples

Example 1: LocalDate - Working with Dates

```
// Get current dateLocalDate today =
LocalDate.now();System.out.println("Today: " + today); // Output:
2024-04-08// Create specific dateLocalDate birthday =
LocalDate.of(1990, 5, 15);System.out.println("Birthday: " + birthday);
// Output: 1990-05-15// Add daysLocalDate nextWeek =
today.plusDays(7);System.out.println("Next week: " + nextWeek);
```

Example 2: LocalTime - Working with Time

```
// Get current timeLocalTime now =
LocalTime.now();System.out.println("Current time: " + now); // Output:
14:30:45.123// Create specific timeLocalTime lunchTime =
LocalTime.of(12, 30);System.out.println("Lunch: " + lunchTime); //
Output: 12:30// Add hoursLocalTime later =
now.plusHours(2);System.out.println("Later: " + later);
```

Example 3: LocalDateTime - Date and Time Together

```
// Get current date and timeLocalDateTime now =
LocalDateTime.now();System.out.println("Now: " + now); // Output:
2024-04-08T14:30:45// Create specific date and timeLocalDateTime
meeting = LocalDateTime.of(2024, 4, 10, 15,
30);System.out.println("Meeting: " + meeting); // Output:
2024-04-10T15:30
```

Example 4: Formatting Dates

```
LocalDate today = LocalDate.now();// Format dateDateTimeFormatter
formatter = DateTimeFormatter.ofPattern("dd-MM-yyyy");String
formattedDate = today.format(formatter);System.out.println("Formatted:
" + formattedDate); // Output: 08-04-2024
```

Example 5: Period - Date Difference

```
LocalDate startDate = LocalDate.of(2020, 1, 1);LocalDate endDate =
LocalDate.of(2024, 4, 8);Period period = Period.between(startDate,
endDate);System.out.println("Years: " + period.getYears());   //
Output: 4System.out.println("Months: " + period.getMonths()); //
Output: 3System.out.println("Days: " + period.getDays());     //
Output: 7
```

Example 6: Duration - Time Difference

```
LocalTime startTime = LocalTime.of(9, 0);LocalTime endTime =
LocalTime.of(17, 30);Duration duration = Duration.between(startTime,
endTime);System.out.println("Hours: " + duration.toHours());     //
Output: 8System.out.println("Minutes: " + duration.toMinutes()); //
Output: 510
```

Interview Questions and Answers

Q1: Why was the new Date and Time API introduced in Java 8?

Answer: The old Date and Calendar classes had several problems: they were not thread-safe, dates were mutable (could be changed), month numbering started from 0, and the API was confusing. The new API (java.time package) is immutable, thread-safe, and much easier to use.

Q2: What is the difference between LocalDate, LocalTime, and LocalDateTime?

- **LocalDate:** Represents only date (year, month, day)
- **LocalTime:** Represents only time (hour, minute, second)

- **LocalDateTime:** Represents both date and time (no time zone)

Q3: What is the difference between Period and Duration?

Answer: Period is used for date-based differences and measures in years, months, and days. Duration is used for time-based differences and measures in hours, minutes, and seconds. Period is used with LocalDate, while Duration is used with LocalTime or LocalDateTime.

Q4: Are the new Date and Time classes thread-safe?

Answer: Yes. All classes in the java.time package are immutable and thread-safe. Once created, they cannot be modified. Any operation that appears to modify them actually returns a new object.

Q5: How do you format a date in Java 8?

Answer: Use DateTimeFormatter class. Create a formatter with a pattern using DateTimeFormatter.ofPattern("dd-MM-yyyy") and then call the format() method on the date object. Example: date.format(formatter)

8. forEach Method

Brief Explanation

The forEach method is a new way to iterate over collections in Java 8. It takes a lambda expression or method reference and executes it for each element in the collection. It makes code cleaner and more concise than traditional for loops.

Summary

- Added to Iterable interface
- Works with all collections (List, Set, etc.)
- Takes a Consumer functional interface
- Makes iteration simple and readable
- Syntax: collection.forEach(element -> action)

Detailed Description

The forEach method is a default method added to the Iterable interface in Java 8. It takes a Consumer functional interface as a parameter, which means you can pass a lambda expression or method reference to it. This makes iterating over collections much simpler and more readable than traditional for loops.

Simple Examples

Example 1: Basic forEach with List

Old Way:

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");for
(String name : names) {    System.out.println(name);}
```

New Way (with forEach):

```
List<String> names = Arrays.asList("Alice", "Bob",
"Charlie");names.forEach(name -> System.out.println(name));// Even
simpler with method referencenames.forEach(System.out::println);
```

Example 2: forEach with Numbers

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);// Print each
numbernumbers.forEach(num -> System.out.println(num));// Print each
number multiplied by 2numbers.forEach(num -> System.out.println(num *
2));
```

Example 3: forEach with Set

```
Set<String> fruits = new
HashSet<>();fruits.add("Apple");fruits.add("Banana");fruits.add("Orang
e");// Print each fruitfruits.forEach(fruit ->
System.out.println(fruit));
```

Example 4: forEach with Map

```
Map<String, Integer> scores = new HashMap<>();scores.put("Alice",
90);scores.put("Bob", 85);scores.put("Charlie", 95);// Print each
key-value pairscores.forEach((name, score) ->
System.out.println(name + ": " + score));
```

Example 5: forEach with Custom Objects

```
class Employee {    String name;    int age;        Employee(String
name, int age) {        this.name = name;        this.age = age;
}}List<Employee> employees = Arrays.asList(    new Employee("Alice",
25),    new Employee("Bob", 30),    new Employee("Charlie", 28));//
Print each employee's detailsemployees.forEach(emp ->
System.out.println(emp.name + " is " + emp.age + " years old"));
```

Interview Questions and Answers

Q1: What is forEach method in Java 8?

Answer: forEach is a default method added to the Iterable interface in Java 8. It takes a Consumer functional interface as a parameter and executes the given action for each element in the collection. It's a cleaner alternative to traditional for loops.

Q2: Can we use forEach with Map?

Answer: Yes. Map has its own forEach method that takes a BiConsumer (accepts two parameters: key and value). Example: map.forEach((key, value) -> System.out.println(key + ": " + value));

Q3: What is the difference between forEach and for loop?

Answer: forEach is more concise and readable. It's a functional approach that works with lambda expressions. However, you cannot use break or continue statements with forEach like you can with traditional for loops. Also, forEach cannot modify the collection while iterating (it will throw ConcurrentModificationException).

Q4: Can we use forEach with Arrays?

Answer: Yes. You can convert an array to a stream and then use forEach. Example: Arrays.stream(array).forEach(element -> System.out.println(element));

Q5: Can we modify elements while using forEach?

Answer: You can modify the elements themselves (if they are mutable objects), but you cannot add or remove elements from the collection while iterating. Doing so will throw a ConcurrentModificationException.

End of Java 8 Interview Preparation Guide

Practice these examples and you'll be ready for your interview!